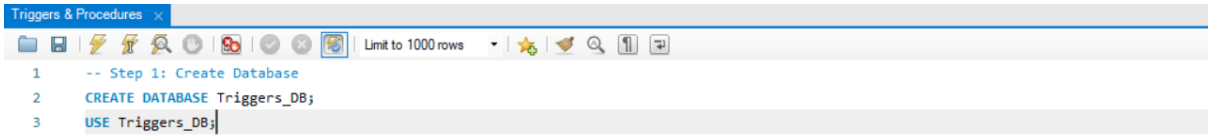
Triggers & Stored Procedures

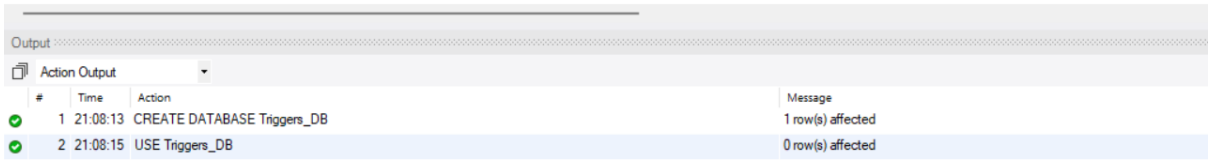
Ili.Harshika (2520030607)

Question :

Develop a Bank Database using Customer, Account and Bank_Transaction tables. Implement stored procedures for customer and account operations, deposits, withdrawals, money transfer and balance retrieval. Implement BEFORE and AFTER triggers for balance validation, transaction validation, transaction auditing, automatic balance updates and insufficient-balance prevention.

Step 1 : Create Database and Table





Triggers & Procedures

Limit to 1000 rows

```sql
-- Step 1: Create Database
CREATE DATABASE Triggers_DB;
USE Triggers_DB;
-- Step 2: Create Customer Table
CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15),
    Email VARCHAR(100),
    City VARCHAR(50)
);
```

Output

Action Output

#	Time	Action	Message
1	21:08:13	CREATE DATABASE Triggers_DB	1 row(s) affected
2	21:08:15	USE Triggers_DB	0 row(s) affected
3	21:08:37	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NUL...	0 row(s) affected

Triggers & Procedures

Limit to 1000 rows

```sql
-- Step 1: Create Database
CREATE DATABASE Triggers_DB;
USE Triggers_DB;
-- Step 2: Create Customer Table
CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15),
    Email VARCHAR(100),
    City VARCHAR(50)
);
-- Step 3: Create Account Table
CREATE TABLE Account (
    Account_No INT PRIMARY KEY,
    Customer_ID INT,
    Account_Type VARCHAR(20),
    Balance DECIMAL(12,2) DEFAULT 0,
    Branch VARCHAR(50),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);
```

Output

Action Output

#	Time	Action	Message
1	21:08:13	CREATE DATABASE Triggers_DB	1 row(s) affected
2	21:08:15	USE Triggers_DB	0 row(s) affected
3	21:08:37	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NUL...	0 row(s) affected
4	21:08:58	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARCHA...	0 row(s) affected

Triggers & Procedures

```
10      City VARCHAR(50)
11  );
12  -- Step 3: Create Account Table
13  CREATE TABLE Account (
14      Account_No INT PRIMARY KEY,
15      Customer_ID INT,
16      Account_Type VARCHAR(20),
17      Balance DECIMAL(12,2) DEFAULT 0,
18      Branch VARCHAR(50),
19      FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
20  );
21  -- Step 4: Create Bank Transaction Table
22  CREATE TABLE Bank_Transaction (
23      Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
24      Account_No INT,
25      Transaction_Type VARCHAR(20),
26      Amount DECIMAL(12,2),
27      Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
28      FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
29  );
```

Output

Action Output

#	Time	Action	Message
1	21:08:13	CREATE DATABASE Triggers_DB	1 row(s) affected
2	21:08:15	USE Triggers_DB	0 row(s) affected
3	21:08:37	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NUL...	0 row(s) affected
4	21:08:58	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARCHA...	0 row(s) affected
5	21:09:28	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_N...	0 row(s) affected

Triggers & Procedures

```
17      Balance DECIMAL(12,2) DEFAULT 0,
18      Branch VARCHAR(50),
19      FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
20  );
21  -- Step 4: Create Bank Transaction Table
22  CREATE TABLE Bank_Transaction (
23      Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
24      Account_No INT,
25      Transaction_Type VARCHAR(20),
26      Amount DECIMAL(12,2),
27      Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
28      FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
29  );
30  -- Step 5: Insert Customer Data
31  INSERT INTO Customer VALUES
32  (101,'Harshika','9876543210','harshika@gmail.com','Hyderabad'),
33  (102,'Ravinder','9876543211','ravi@gmail.com','Vijayawada'),
34  (103,'Santhvin','9876543212','santhvin@gmail.com','Bangalore'),
35  (104,'Srilatha','9876543213','srilatha@gmail.com','Chennai'),
36  (105,'Varshika','9876543214','varshika@gmail.com','Hyderabad');
```

Output

Action Output

#	Time	Action	Message
1	21:08:13	CREATE DATABASE Triggers_DB	1 row(s) affected
2	21:08:15	USE Triggers_DB	0 row(s) affected
3	21:08:37	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NUL...	0 row(s) affected
4	21:08:58	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARCHA...	0 row(s) affected
5	21:09:28	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_N...	0 row(s) affected
6	21:11:10	INSERT INTO Customer VALUES (101,'Harshika','9876543210','harshika@gmail.com','Hyderabad'), (102,'Ravind...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

Triggers & Procedures

```
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
);
-- Step 5: Insert Customer Data
INSERT INTO Customer VALUES
(101,'Harshika','9876543210','harshika@gmail.com','Hyderabad'),
(102,'Ravinder','9876543211','ravi@gmail.com','Vijayawada'),
(103,'Santhvin','9876543212','santhvin@gmail.com','Bangalore'),
(104,'Srilatha','9876543213','srilatha@gmail.com','Chennai'),
(105,'Varshika','9876543214','varshika@gmail.com','Hyderabad');
-- Step 6: Insert Account Data
INSERT INTO Account VALUES
(10001,101,'Savings',50000,'Hyderabad'),
(10002,102,'Savings',75000,'Vijayawada'),
(10003,103,'Current',120000,'Bangalore'),
(10004,104,'Savings',45000,'Chennai'),
(10005,105,'Current',90000,'Hyderabad');
```

Output

Action Output

#	Time	Action	Message
2	21:08:15	USE Triggers_DB	0 row(s) affected
3	21:08:37	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NU...	0 row(s) affected
4	21:08:58	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARC...	0 row(s) affected
5	21:09:28	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_...	0 row(s) affected
6	21:11:10	INSERT INTO Customer VALUES (101,'Harshika','9876543210','harshika@gmail.com','Hyderabad'), (102,'Ravin...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
7	21:11:34	INSERT INTO Account VALUES (10001,101,'Savings',50000,'Hyderabad'), (10002,102,'Savings',75000,'Vijaya...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

Triggers & Procedures

```
(102,'Ravinder','9876543211','ravi@gmail.com','Vijayawada'),
(103,'Santhvin','9876543212','santhvin@gmail.com','Bangalore'),
(104,'Srilatha','9876543213','srilatha@gmail.com','Chennai'),
(105,'Varshika','9876543214','varshika@gmail.com','Hyderabad');
-- Step 6: Insert Account Data
INSERT INTO Account VALUES
(10001,101,'Savings',50000,'Hyderabad'),
(10002,102,'Savings',75000,'Vijayawada'),
(10003,103,'Current',120000,'Bangalore'),
(10004,104,'Savings',45000,'Chennai'),
(10005,105,'Current',90000,'Hyderabad');
-- Step 7: Insert Transaction Data
INSERT INTO Bank_Transaction
(Account_No,Transaction_Type,Amount)
VALUES
(10001,'DEPOSIT',10000),
(10002,'DEPOSIT',15000),
(10003,'WITHDRAW',20000),
(10004,'DEPOSIT',5000),
(10005,'WITHDRAW',10000);
```

Output

Action Output

#	Time	Action	Message
3	21:08:37	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NU...	0 row(s) affected
4	21:08:58	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARC...	0 row(s) affected
5	21:09:28	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_...	0 row(s) affected
6	21:11:10	INSERT INTO Customer VALUES (101,'Harshika','9876543210','harshika@gmail.com','Hyderabad'), (102,'Ravin...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
7	21:11:34	INSERT INTO Account VALUES (10001,101,'Savings',50000,'Hyderabad'), (10002,102,'Savings',75000,'Vijaya...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
8	21:11:55	INSERT INTO Bank_Transaction (Account_No,Transaction_Type,Amount) VALUES (10001,'DEPOSIT',10000)...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

Triggers & Procedures

```
    (Account_No,Transaction_Type,Amount)
    VALUES
    (10001,'DEPOSIT',10000),
    (10002,'DEPOSIT',15000),
    (10003,'WITHDRAW',20000),
    (10004,'DEPOSIT',5000),
    (10005,'WITHDRAW',10000);
    -- Step 8: Display Table Data
    SELECT * FROM Customer;
```

Result Grid

Customer_ID	Customer_Name	Phone	Email	City
101	Harshika	9876543210	harshika@gmail.com	Hyderabad
102	Ravinder	9876543211	ravi@gmail.com	Vijayawada
103	Santhvin	9876543212	santhvin@gmail.com	Bangalore
104	Srilatha	9876543213	srilatha@gmail.com	Chennai
105	Varshika	9876543214	varshika@gmail.com	Hyderabad
NULL	NULL	NULL	NULL	NULL

Customer 1

Output

Action Output

#	Time	Action	Message
4	21:08:58	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARC...	0 row(s) affected
5	21:09:28	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account...	0 row(s) affected
6	21:11:10	INSERT INTO Customer VALUES (101,'Harshika','9876543210','harshika@gmail.com','Hyderabad'), (102,'Ravi...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
7	21:11:34	INSERT INTO Account VALUES (10001,101,'Savings',50000,'Hyderabad'), (10002,102,'Savings',75000,'Vijay...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
8	21:11:55	INSERT INTO Bank_Transaction (Account_No,Transaction_Type,Amount) VALUES (10001,'DEPOSIT',1000...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
9	21:12:22	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned

Triggers & Procedures

```
    VALUES
    (10001,'DEPOSIT',10000),
    (10002,'DEPOSIT',15000),
    (10003,'WITHDRAW',20000),
    (10004,'DEPOSIT',5000),
    (10005,'WITHDRAW',10000);
    -- Step 8: Display Table Data
    SELECT * FROM Customer;
    SELECT * FROM Account;
```

Result Grid

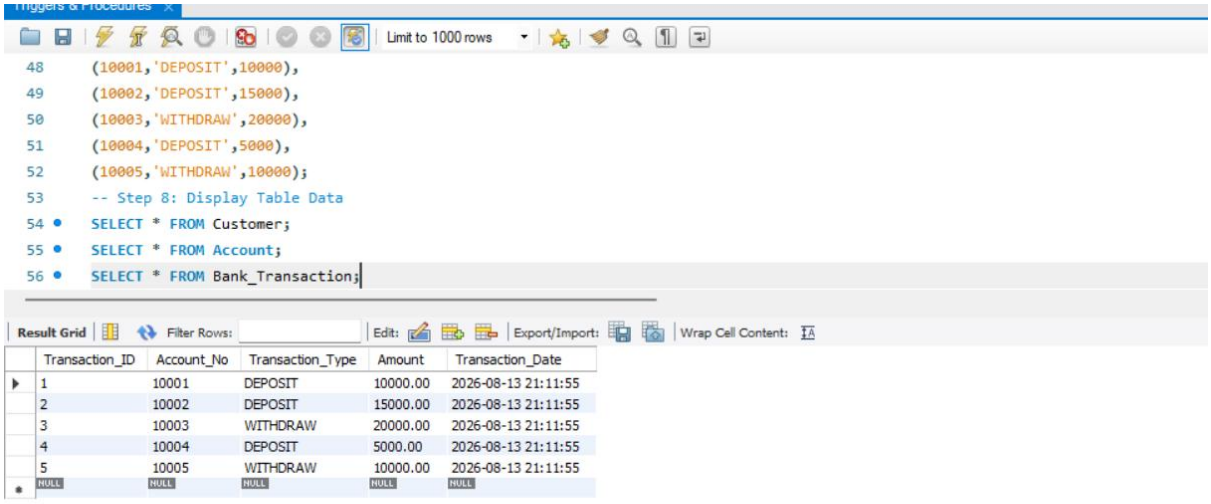
Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
NULL	NULL	NULL	NULL	NULL

Account 2

Output

Action Output

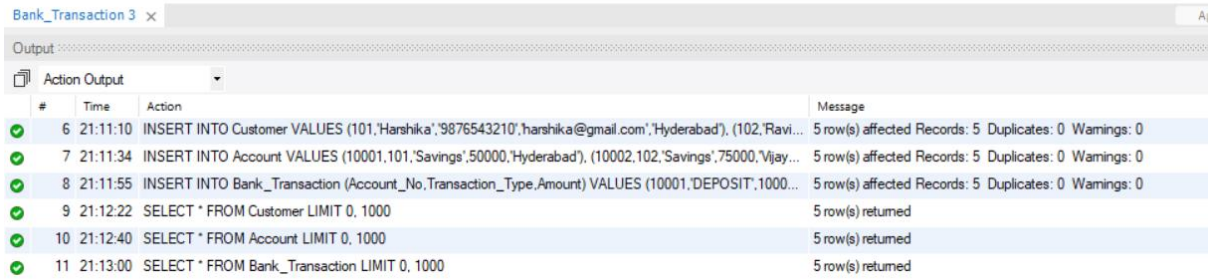
#	Time	Action	Message
5	21:09:28	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account...	0 row(s) affected
6	21:11:10	INSERT INTO Customer VALUES (101,'Harshika','9876543210','harshika@gmail.com','Hyderabad'), (102,'Ravi...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
7	21:11:34	INSERT INTO Account VALUES (10001,101,'Savings',50000,'Hyderabad'), (10002,102,'Savings',75000,'Vijay...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
8	21:11:55	INSERT INTO Bank_Transaction (Account_No,Transaction_Type,Amount) VALUES (10001,'DEPOSIT',1000...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
9	21:12:22	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
10	21:12:40	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned



```
48      (10001,'DEPOSIT',10000),
49      (10002,'DEPOSIT',15000),
50      (10003,'WITHDRAW',20000),
51      (10004,'DEPOSIT',5000),
52      (10005,'WITHDRAW',10000);
53      -- Step 8: Display Table Data
54  •   SELECT * FROM Customer;
55  •   SELECT * FROM Account;
56  •   SELECT * FROM Bank_Transaction;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
1	10001	DEPOSIT	10000.00	2026-08-13 21:11:55
2	10002	DEPOSIT	15000.00	2026-08-13 21:11:55
3	10003	WITHDRAW	20000.00	2026-08-13 21:11:55
4	10004	DEPOSIT	5000.00	2026-08-13 21:11:55
5	10005	WITHDRAW	10000.00	2026-08-13 21:11:55
NULL	NULL	NULL	NULL	NULL



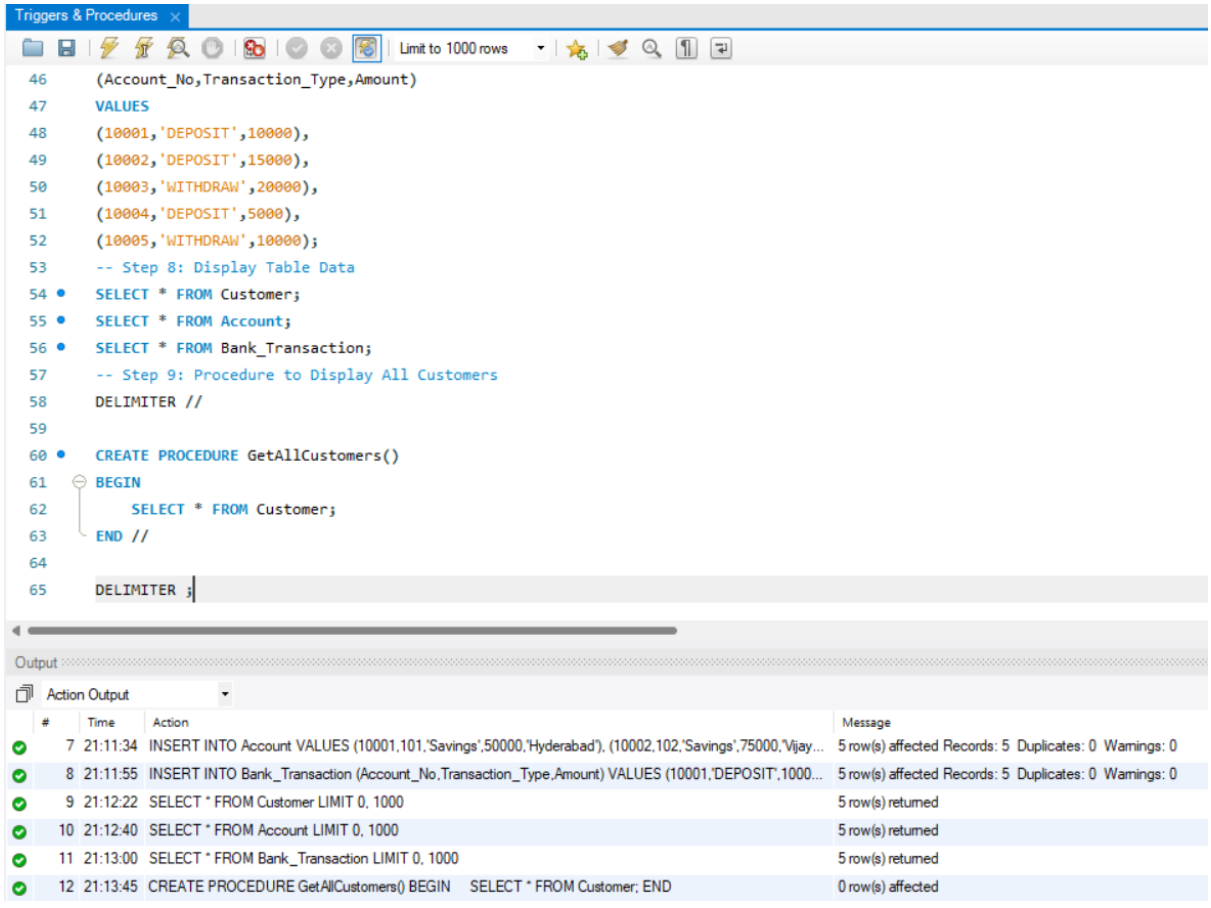
Bank_Transaction 3

Output

Action Output

#	Time	Action	Message
6	21:11:10	INSERT INTO Customer VALUES (101,'Harshika','9876543210','harshika@gmail.com','Hyderabad'), (102,'Ravi...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
7	21:11:34	INSERT INTO Account VALUES (10001,101,'Savings',50000,'Hyderabad'), (10002,102,'Savings',75000,'Vijay...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
8	21:11:55	INSERT INTO Bank_Transaction (Account_No,Transaction_Type,Amount) VALUES (10001,'DEPOSIT',1000...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
9	21:12:22	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
10	21:12:40	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
11	21:13:00	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned

Step 2 : STORED PROCEDURES



Triggers & Procedures

```
46      (Account_No,Transaction_Type,Amount)
47      VALUES
48      (10001,'DEPOSIT',10000),
49      (10002,'DEPOSIT',15000),
50      (10003,'WITHDRAW',20000),
51      (10004,'DEPOSIT',5000),
52      (10005,'WITHDRAW',10000);
53      -- Step 8: Display Table Data
54  •   SELECT * FROM Customer;
55  •   SELECT * FROM Account;
56  •   SELECT * FROM Bank_Transaction;
57      -- Step 9: Procedure to Display All Customers
58      DELIMITER //
59
60  •   CREATE PROCEDURE GetAllCustomers()
61      BEGIN
62          SELECT * FROM Customer;
63      END //
64
65      DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
7	21:11:34	INSERT INTO Account VALUES (10001,101,'Savings',50000,'Hyderabad'), (10002,102,'Savings',75000,'Vijay...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
8	21:11:55	INSERT INTO Bank_Transaction (Account_No,Transaction_Type,Amount) VALUES (10001,'DEPOSIT',1000...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
9	21:12:22	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
10	21:12:40	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
11	21:13:00	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
12	21:13:45	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END	0 row(s) affected

Triggers & Procedures

```
59
60 •  CREATE PROCEDURE GetAllCustomers()
61    BEGIN
62        SELECT * FROM Customer;
63    END //
64
65    DELIMITER ;
66 •  -- Execute Step 9
67    CALL GetAllCustomers();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Customer_ID	Customer_Name	Phone	Email	City
101	Harshika	9876543210	harshika@gmail.com	Hyderabad
102	Ravinder	9876543211	ravi@gmail.com	Vijayawada
103	Santhvin	9876543212	santhvin@gmail.com	Bangalore
104	Srilatha	9876543213	srilatha@gmail.com	Chennai
105	Varshika	9876543214	varshika@gmail.com	Hyderabad

Result 4

Output

Action Output

#	Time	Action	Message
8	21:11:55	INSERT INTO Bank_Transaction (Account_No,Transaction_Type,Amount) VALUES (10001,'DEPOSIT',1000...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
9	21:12:22	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
10	21:12:40	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
11	21:13:00	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
12	21:13:45	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END	0 row(s) affected
13	21:14:10	CALL GetAllCustomers()	5 row(s) returned

Triggers & Procedures

```
59
60 •  CREATE PROCEDURE GetAllCustomers()
61    BEGIN
62        SELECT * FROM Customer;
63    END //
64
65    DELIMITER ;
66 •  -- Execute Step 9
67    CALL GetAllCustomers();
68    -- Step 10: Procedure to Display Account Details
69    DELIMITER //
70
71 •  CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT)
72    BEGIN
73        SELECT *
74        FROM Account
75        WHERE Account_No = p_Account_No;
76    END //
77
78    DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
9	21:12:22	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
10	21:12:40	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
11	21:13:00	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
12	21:13:45	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END	0 row(s) affected
13	21:14:10	CALL GetAllCustomers()	5 row(s) returned
14	21:14:36	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account ...	0 row(s) affected

Triggers & Procedures

```
77
78      DELIMITER ;
79      -- Step 11: Procedure to Find Customer Accounts
80      DELIMITER //
81
82      CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT)
83      BEGIN
84          SELECT C.Customer_ID,
85                 C.Customer_Name,
86                 A.Account_No,
87                 A.Account_Type,
88                 A.Balance,
89                 A.Branch
90          FROM Customer C
91          JOIN Account A
92          ON C.Customer_ID = A.Customer_ID
93          WHERE C.Customer_ID = p_Customer_ID;
94      END //
95
96      DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
10	21:12:40	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
11	21:13:00	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
12	21:13:45	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END	0 row(s) affected
13	21:14:10	CALL GetAllCustomers()	5 row(s) returned
14	21:14:36	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account ...	0 row(s) affected
15	21:15:01	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected

Triggers & Procedures

```
90          FROM Customer C
91          JOIN Account A
92          ON C.Customer_ID = A.Customer_ID
93          WHERE C.Customer_ID = p_Customer_ID;
94      END //
95
96      DELIMITER ;
97      -- Execute Step 11
98      CALL GetCustomerAccounts(101);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Harshika	10001	Savings	50000.00	Hyderabad

Result 5

Output

Action Output

#	Time	Action	Message
11	21:13:00	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
12	21:13:45	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END	0 row(s) affected
13	21:14:10	CALL GetAllCustomers()	5 row(s) returned
14	21:14:36	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account ...	0 row(s) affected
15	21:15:01	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
16	21:15:37	CALL GetCustomerAccounts(101)	1 row(s) returned

Triggers & Procedures

```
93      WHERE C.Customer_ID = p_Customer_ID;
94  END //
95
96  DELIMITER ;
97  -- Execute Step 11
98  CALL GetCustomerAccounts(101);
99  -- Step 12: Procedure to Deposit Money
100 DELIMITER //
101
102 CREATE PROCEDURE DepositMoney(
103     IN p_Account_No INT,
104     IN p_Amount DECIMAL(12,2)
105 )
106 BEGIN
107     UPDATE Account
108     SET Balance = Balance + p_Amount
109     WHERE Account_No = p_Account_No;
110 END //
111
112 DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
12	21:13:45	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END	0 row(s) affected
13	21:14:10	CALL GetAllCustomers()	5 row(s) returned
14	21:14:36	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account ...	0 row(s) affected
15	21:15:01	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
16	21:15:37	CALL GetCustomerAccounts(101)	1 row(s) returned
17	21:16:02	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected

Triggers & Procedures

```
95
96  DELIMITER ;
97  -- Execute Step 11
98  CALL GetCustomerAccounts(101);
99  -- Step 12: Procedure to Deposit Money
100 DELIMITER //
101
102 CREATE PROCEDURE DepositMoney(
103     IN p_Account_No INT,
104     IN p_Amount DECIMAL(12,2)
105 )
106 BEGIN
107     UPDATE Account
108     SET Balance = Balance + p_Amount
109     WHERE Account_No = p_Account_No;
110 END //
111
112 DELIMITER ;
113 -- Execute Step 12
114 CALL DepositMoney(10001,5000);
```

Output

Action Output

#	Time	Action	Message
13	21:14:10	CALL GetAllCustomers()	5 row(s) returned
14	21:14:36	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account ...	0 row(s) affected
15	21:15:01	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
16	21:15:37	CALL GetCustomerAccounts(101)	1 row(s) returned
17	21:16:02	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
18	21:16:23	CALL DepositMoney(10001,5000)	1 row(s) affected

Triggers & Procedures

```
109         WHERE Account_No = p_Account_No;
110     END //
111
112     DELIMITER ;
113     -- Execute Step 12
114     CALL DepositMoney(10001,5000);
115     -- Step 13: Procedure to Withdraw Money
116     DELIMITER //
117
118     CREATE PROCEDURE WithdrawMoney(
119         IN p_Account_No INT,
120         IN p_Amount DECIMAL(12,2)
121     )
122     BEGIN
123         UPDATE Account
124         SET Balance = Balance - p_Amount
125         WHERE Account_No = p_Account_No;
126     END //
127
128     DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
14	21:14:36	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account ...	0 row(s) affected
15	21:15:01	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
16	21:15:37	CALL GetCustomerAccounts(101)	1 row(s) returned
17	21:16:02	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
18	21:16:23	CALL DepositMoney(10001,5000)	1 row(s) affected
19	21:16:53	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected

Triggers & Procedures

```
111
112     DELIMITER ;
113     -- Execute Step 12
114     CALL DepositMoney(10001,5000);
115     -- Step 13: Procedure to Withdraw Money
116     DELIMITER //
117
118     CREATE PROCEDURE WithdrawMoney(
119         IN p_Account_No INT,
120         IN p_Amount DECIMAL(12,2)
121     )
122     BEGIN
123         UPDATE Account
124         SET Balance = Balance - p_Amount
125         WHERE Account_No = p_Account_No;
126     END //
127
128     DELIMITER ;
129     -- Execute Step 13
130     CALL WithdrawMoney(10001,3000);
```

Output

Action Output

#	Time	Action	Message
15	21:15:01	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
16	21:15:37	CALL GetCustomerAccounts(101)	1 row(s) returned
17	21:16:02	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
18	21:16:23	CALL DepositMoney(10001,5000)	1 row(s) affected
19	21:16:53	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
20	21:17:19	CALL WithdrawMoney(10001,3000)	1 row(s) affected

Step 3 : TRRIGERS

Triggers & Procedures

```
125     WHERE Account_No = p_Account_No;
126 END //
127
128 DELIMITER ;
129 -- Execute Step 13
130 CALL WithdrawMoney(10001,3000);
131 -- Step 14: Trigger to Prevent Negative Balance
132 DELIMITER //
133
134 CREATE TRIGGER CheckBalance
135 BEFORE UPDATE ON Account
136 FOR EACH ROW
137 BEGIN
138     IF NEW.Balance < 0 THEN
139         SIGNAL SQLSTATE '45000'
140         SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
141     END IF;
142 END //
143
144 DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
16	21:15:37	CALL GetCustomerAccounts(101)	1 row(s) returned
17	21:16:02	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
18	21:16:23	CALL DepositMoney(10001,5000)	1 row(s) affected
19	21:16:53	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
20	21:17:19	CALL WithdrawMoney(10001,3000)	1 row(s) affected
21	21:17:55	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected

Triggers & Procedures

```
139         SIGNAL SQLSTATE '45000'
140         SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
141     END IF;
142 END //
143
144 DELIMITER ;
145 -- Step 15: Trigger to Prevent Invalid Transaction Amount
146 DELIMITER //
147
148 CREATE TRIGGER CheckTransactionAmount
149 BEFORE INSERT ON Bank_Transaction
150 FOR EACH ROW
151 BEGIN
152     IF NEW.Amount <= 0 THEN
153         SIGNAL SQLSTATE '45000'
154         SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
155     END IF;
156 END //
157
158 DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
17	21:16:02	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
18	21:16:23	CALL DepositMoney(10001,5000)	1 row(s) affected
19	21:16:53	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
20	21:17:19	CALL WithdrawMoney(10001,3000)	1 row(s) affected
21	21:17:55	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
22	21:18:15	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected

Triggers & Procedures

```
148  CREATE TRIGGER CheckTransactionAmount
149  BEFORE INSERT ON Bank_Transaction
150  FOR EACH ROW
151  BEGIN
152      IF NEW.Amount <= 0 THEN
153          SIGNAL SQLSTATE '45000'
154          SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
155      END IF;
156  END //
157
158  DELIMITER ;
159  -- Step 16: Create Transaction Audit Table
160  CREATE TABLE Transaction_Audit (
161      Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
162      Transaction_ID INT,
163      Account_No INT,
164      Transaction_Type VARCHAR(20),
165      Amount DECIMAL(12,2),
166      Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
167  );
```

Output

Action Output

#	Time	Action	Message
18	21:16:23	CALL DepositMoney(10001,5000)	1 row(s) affected
19	21:16:53	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
20	21:17:19	CALL WithdrawMoney(10001,3000)	1 row(s) affected
21	21:17:55	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
22	21:18:15	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
23	21:18:40	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected

Triggers & Procedures

```
162      Transaction_ID INT,
163      Account_No INT,
164      Transaction_Type VARCHAR(20),
165      Amount DECIMAL(12,2),
166      Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
167  );
168  -- Step 17: Trigger for Transaction Audit
169  DELIMITER //
170
171  CREATE TRIGGER TransactionAudit
172  AFTER INSERT ON Bank_Transaction
173  FOR EACH ROW
174  BEGIN
175      INSERT INTO Transaction_Audit
176      (Transaction_ID,Account_No,Transaction_Type,Amount)
177      VALUES
178      (NEW.Transaction_ID,NEW.Account_No,NEW.Transaction_Type,NEW.Amount);
179  END //
180
181  DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
19	21:16:53	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
20	21:17:19	CALL WithdrawMoney(10001,3000)	1 row(s) affected
21	21:17:55	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
22	21:18:15	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
23	21:18:40	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
24	21:19:03	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected

Triggers & Procedures

Limit to 1000 rows

```
183    DELIMITER //
184
185 •  CREATE TRIGGER UpdateBalanceAfterTransaction
186    AFTER INSERT ON Bank_Transaction
187    FOR EACH ROW
188    BEGIN
189        IF NEW.Transaction_Type = 'DEPOSIT' THEN
190            UPDATE Account
191            SET Balance = Balance + NEW.Amount
192            WHERE Account_No = NEW.Account_No;
193
194        ELSEIF NEW.Transaction_Type = 'WITHDRAW' THEN
195            UPDATE Account
196            SET Balance = Balance - NEW.Amount
197            WHERE Account_No = NEW.Account_No;
198        END IF;
199    END //
200
201    DELIMITER ;
202
```

Output

Action Output

#	Time	Action	Message
20	21:17:19	CALL WithdrawMoney(10001,3000)	1 row(s) affected
21	21:17:55	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
22	21:18:15	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
23	21:18:40	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
24	21:19:03	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
25	21:20:20	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected

Triggers & Procedures

Limit to 1000 rows

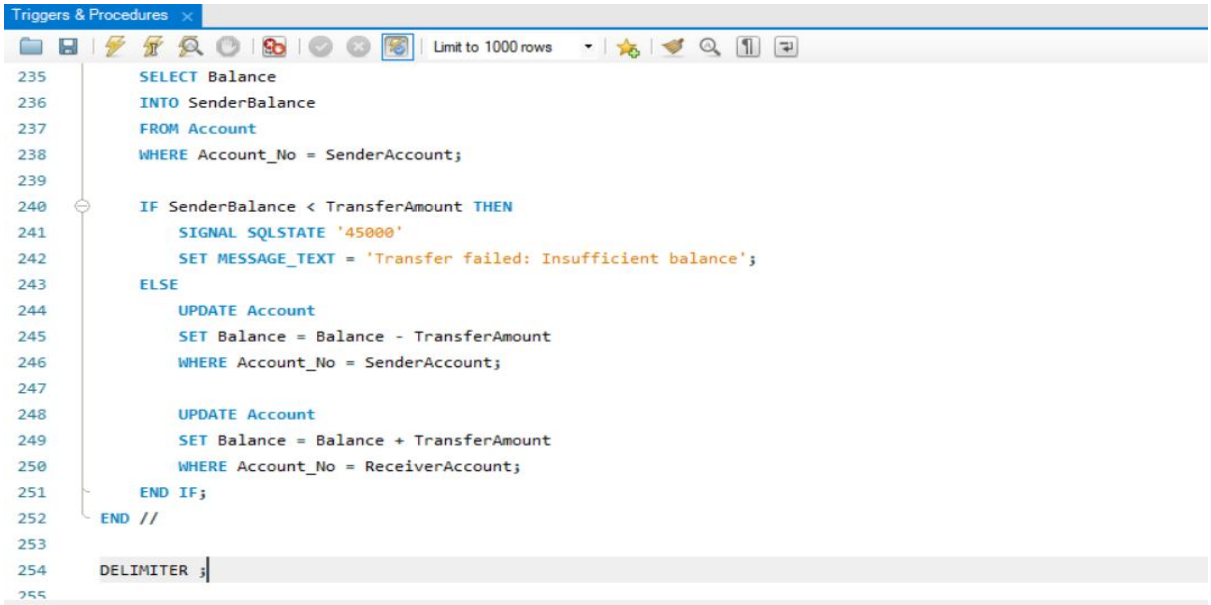
```
204
205 •  CREATE TRIGGER PreventInsufficientWithdrawal
206    BEFORE INSERT ON Bank_Transaction
207    FOR EACH ROW
208    BEGIN
209        DECLARE CurrentBalance DECIMAL(12,2);
210
211        SELECT Balance
212        INTO CurrentBalance
213        FROM Account
214        WHERE Account_No = NEW.Account_No;
215
216        IF NEW.Transaction_Type = 'WITHDRAW'
217           AND NEW.Amount > CurrentBalance THEN
218            SIGNAL SQLSTATE '45000'
219            SET MESSAGE_TEXT = 'Withdrawal failed: Insufficient balance';
220        END IF;
221    END //
222
223    DELIMITER ;
224
```

Output

Action Output

#	Time	Action	Message
21	21:17:55	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
22	21:18:15	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
23	21:18:40	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
24	21:19:03	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
25	21:20:20	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
26	21:20:41	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected

Step 4 : ADVANCED PROCEDURES

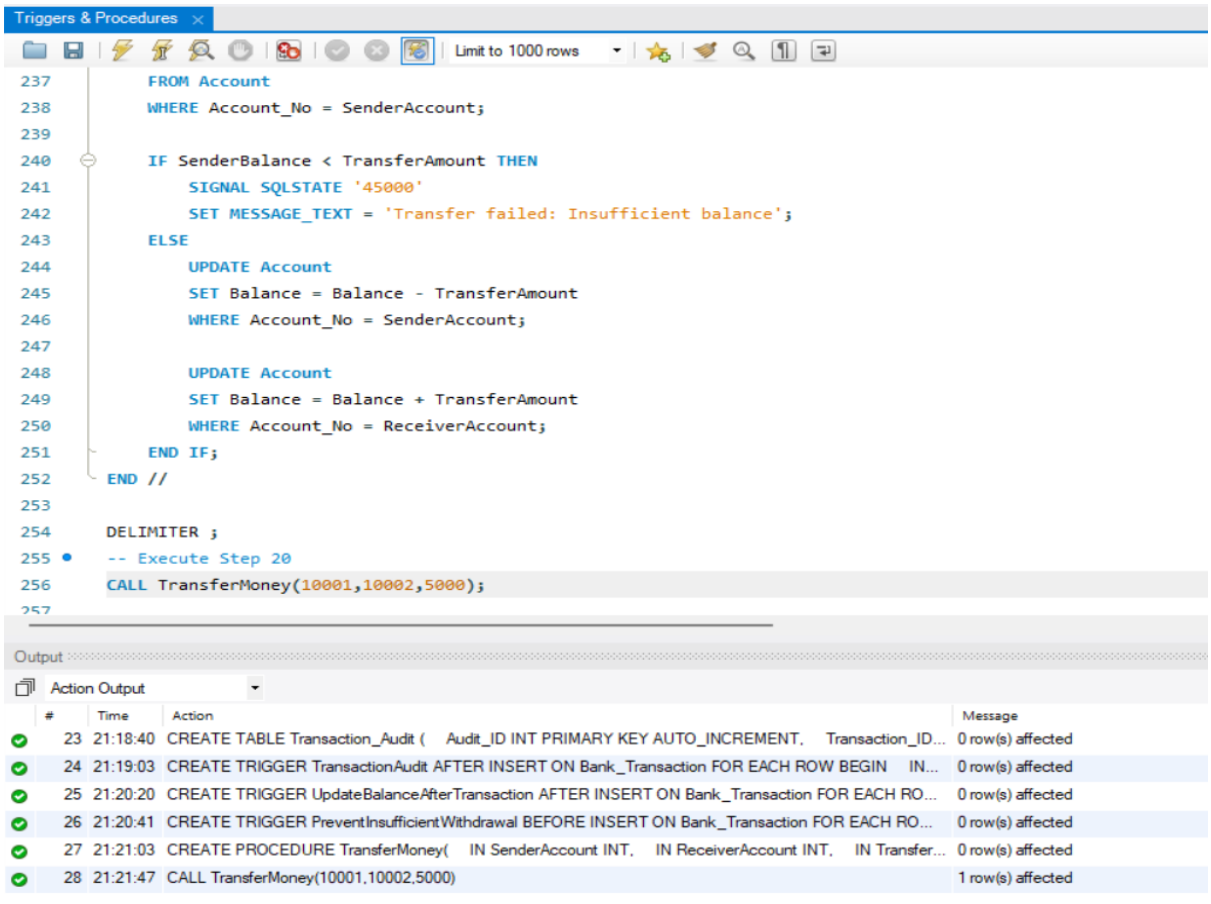


```
235     SELECT Balance
236     INTO SenderBalance
237     FROM Account
238     WHERE Account_No = SenderAccount;
239
240     IF SenderBalance < TransferAmount THEN
241         SIGNAL SQLSTATE '45000'
242         SET MESSAGE_TEXT = 'Transfer failed: Insufficient balance';
243     ELSE
244         UPDATE Account
245         SET Balance = Balance - TransferAmount
246         WHERE Account_No = SenderAccount;
247
248         UPDATE Account
249         SET Balance = Balance + TransferAmount
250         WHERE Account_No = ReceiverAccount;
251     END IF;
252 END //
253
254 DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
22	21:18:15	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
23	21:18:40	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
24	21:19:03	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
25	21:20:20	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
26	21:20:41	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
27	21:21:03	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected



```
237     FROM Account
238     WHERE Account_No = SenderAccount;
239
240     IF SenderBalance < TransferAmount THEN
241         SIGNAL SQLSTATE '45000'
242         SET MESSAGE_TEXT = 'Transfer failed: Insufficient balance';
243     ELSE
244         UPDATE Account
245         SET Balance = Balance - TransferAmount
246         WHERE Account_No = SenderAccount;
247
248         UPDATE Account
249         SET Balance = Balance + TransferAmount
250         WHERE Account_No = ReceiverAccount;
251     END IF;
252 END //
253
254 DELIMITER ;
255 -- Execute Step 20
256 CALL TransferMoney(10001,10002,5000);
```

Output

Action Output

#	Time	Action	Message
23	21:18:40	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
24	21:19:03	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
25	21:20:20	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
26	21:20:41	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
27	21:21:03	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
28	21:21:47	CALL TransferMoney(10001,10002,5000)	1 row(s) affected

Triggers & Procedures

```
251         END IF;
252     END //
253
254     DELIMITER ;
255     -- Execute Step 20
256     CALL TransferMoney(10001,10002,5000);
257     -- Step 21: Procedure for High Balance Accounts
258     DELIMITER //
259
260     CREATE PROCEDURE HighBalanceAccounts(
261         IN MinimumBalance DECIMAL(12,2)
262     )
263     BEGIN
264         SELECT *
265         FROM Account
266         WHERE Balance >= MinimumBalance
267         ORDER BY Balance DESC;
268     END //
269
270     DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
24	21:19:03	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
25	21:20:20	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
26	21:20:41	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
27	21:21:03	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
28	21:21:47	CALL TransferMoney(10001,10002,5000)	1 row(s) affected
29	21:22:12	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected

Triggers & Procedures

```
264         SELECT *
265         FROM Account
266         WHERE Balance >= MinimumBalance
267         ORDER BY Balance DESC;
268     END //
269
270     DELIMITER ;
271     -- Execute Step 21
272     CALL HighBalanceAccounts(50000);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Account_No	Customer_ID	Account_Type	Balance	Branch
10003	103	Current	120000.00	Bangalore
10005	105	Current	90000.00	Hyderabad
10002	102	Savings	80000.00	Vijayawada

Result 6

Output

Action Output

#	Time	Action	Message
25	21:20:20	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
26	21:20:41	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
27	21:21:03	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
28	21:21:47	CALL TransferMoney(10001,10002,5000)	1 row(s) affected
29	21:22:12	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
30	21:22:36	CALL HighBalanceAccounts(50000)	3 row(s) returned

Triggers & Procedures

Limit to 1000 rows

```
269
270     DELIMITER ;
271     -- Execute Step 21
272     CALL HighBalanceAccounts(50000);
273     -- Step 22: Procedure with IN and OUT Parameters
274     DELIMITER //
275
276     CREATE PROCEDURE GetBalance(
277         IN p_Account_No INT,
278         OUT p_Balance DECIMAL(12,2)
279     )
280     BEGIN
281         SELECT Balance
282         INTO p_Balance
283         FROM Account
284         WHERE Account_No = p_Account_No;
285     END //
286
287     DELIMITER ;
288
```

Output

Action Output

#	Time	Action	Message
26	21:20:41	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
27	21:21:03	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
28	21:21:47	CALL TransferMoney(10001,10002,5000)	1 row(s) affected
29	21:22:12	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
30	21:22:36	CALL HighBalanceAccounts(50000)	3 row(s) returned
31	21:23:06	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN ...	0 row(s) affected

Triggers & Procedures

Limit to 1000 rows

```
283         FROM Account
284         WHERE Account_No = p_Account_No;
285     END //
286
287     DELIMITER ;
288     -- Execute Step 22
289     CALL GetBalance(10001,@CurrentBalance);
290
291     SELECT @CurrentBalance;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

@CurrentBalance
NULL

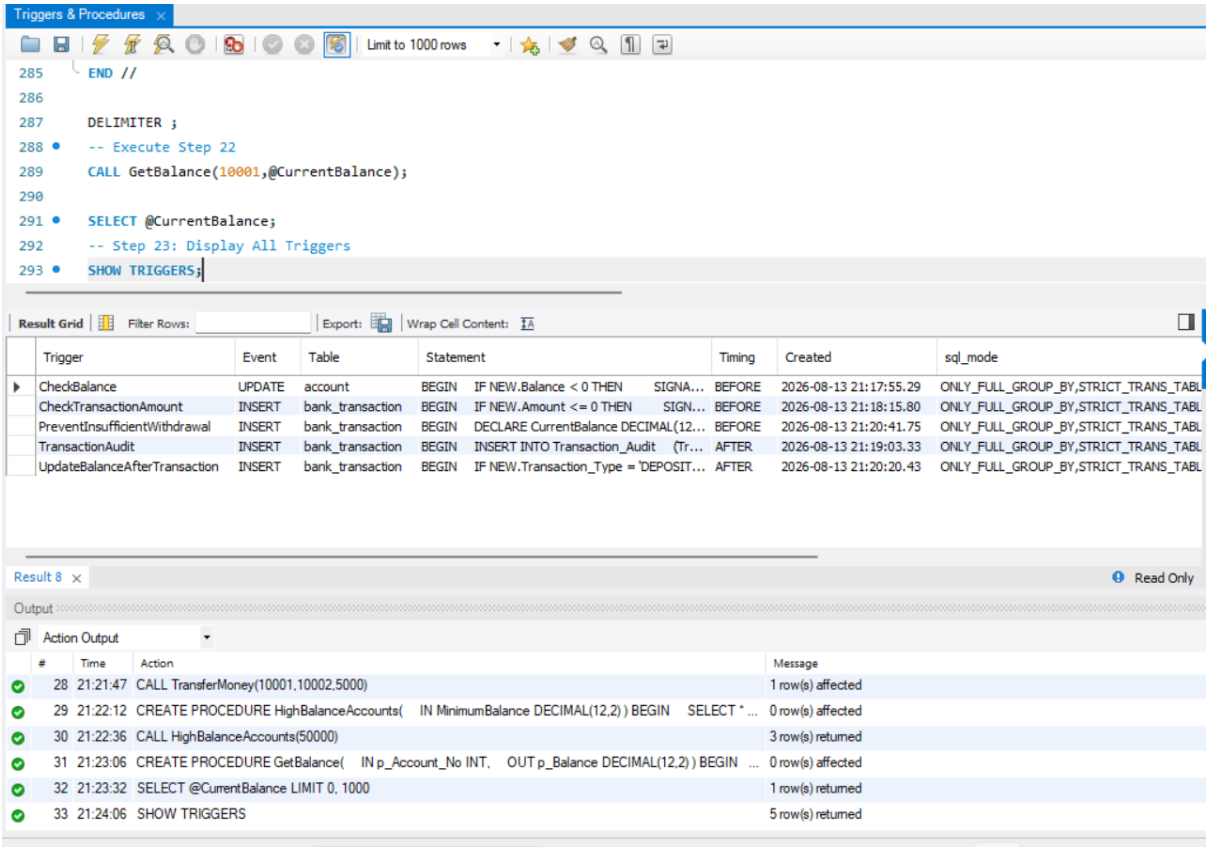
Result 7

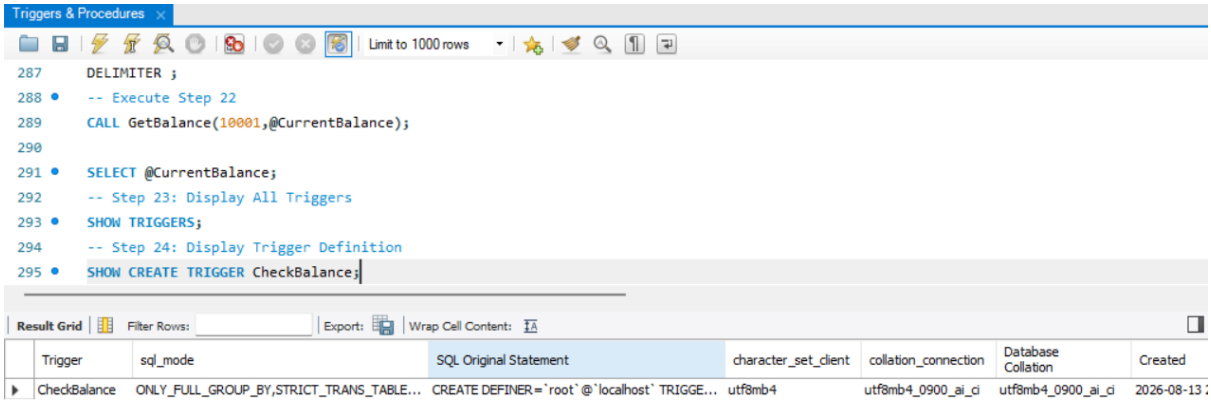
Output

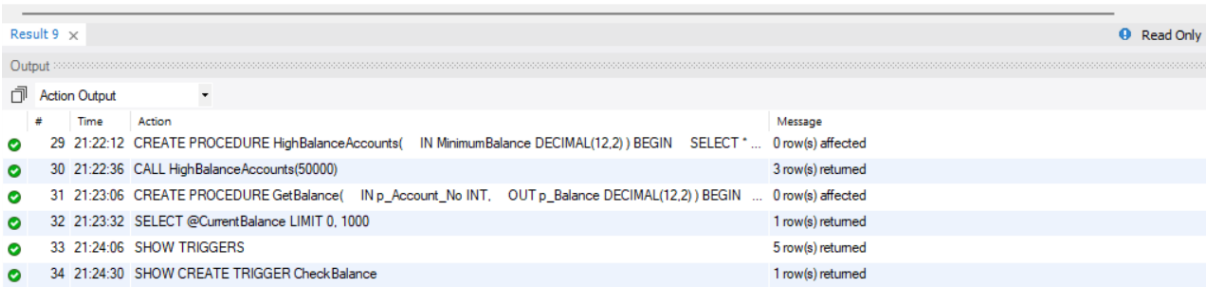
Action Output

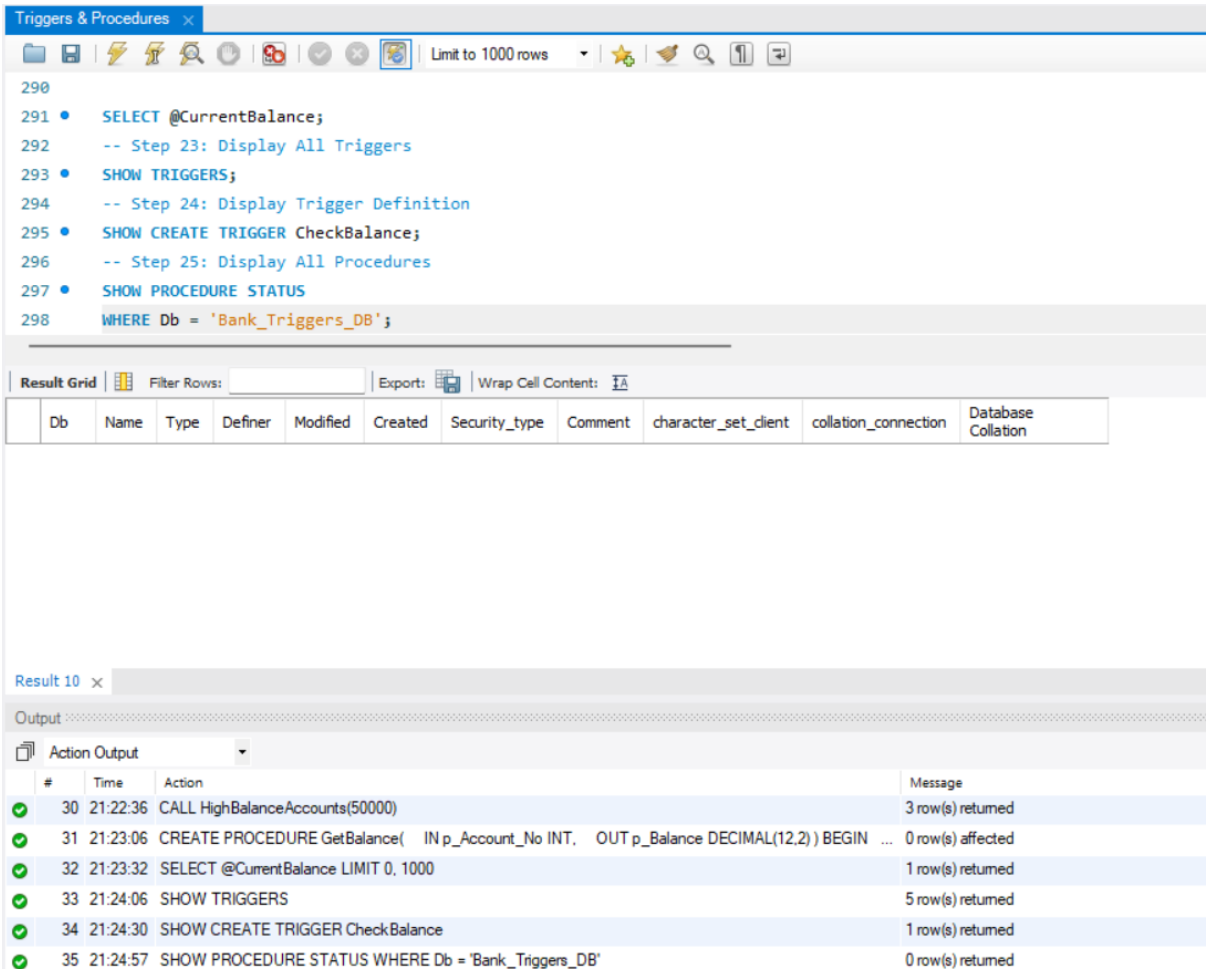
#	Time	Action	Message
27	21:21:03	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
28	21:21:47	CALL TransferMoney(10001,10002,5000)	1 row(s) affected
29	21:22:12	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
30	21:22:36	CALL HighBalanceAccounts(50000)	3 row(s) returned
31	21:23:06	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN ...	0 row(s) affected
32	21:23:32	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned

Step 5 : Management









CONCEPT : PROCEDURES

QUESTION 4:

Create a procedure to display accounts whose balance is greater than a given amount.

Example:

CALL HighBalanceAccounts(100000);

Procedures(Question_4)

Limit to 1000 rows

```sql
-- Step 1: Remove Existing Procedure
DROP PROCEDURE IF EXISTS HighBalanceAccounts;
```

Output

Action Output

#	Time	Action	Message
1	21:45:46	DROP PROCEDURE IF EXISTS HighBalanceAccounts	0 row(s) affected

Procedures(Question_4)

Limit to 1000 rows

```sql
-- Step 1: Remove Existing Procedure
DROP PROCEDURE IF EXISTS HighBalanceAccounts;
-- Step 2: Create Procedure for High Balance Accounts
DELIMITER //

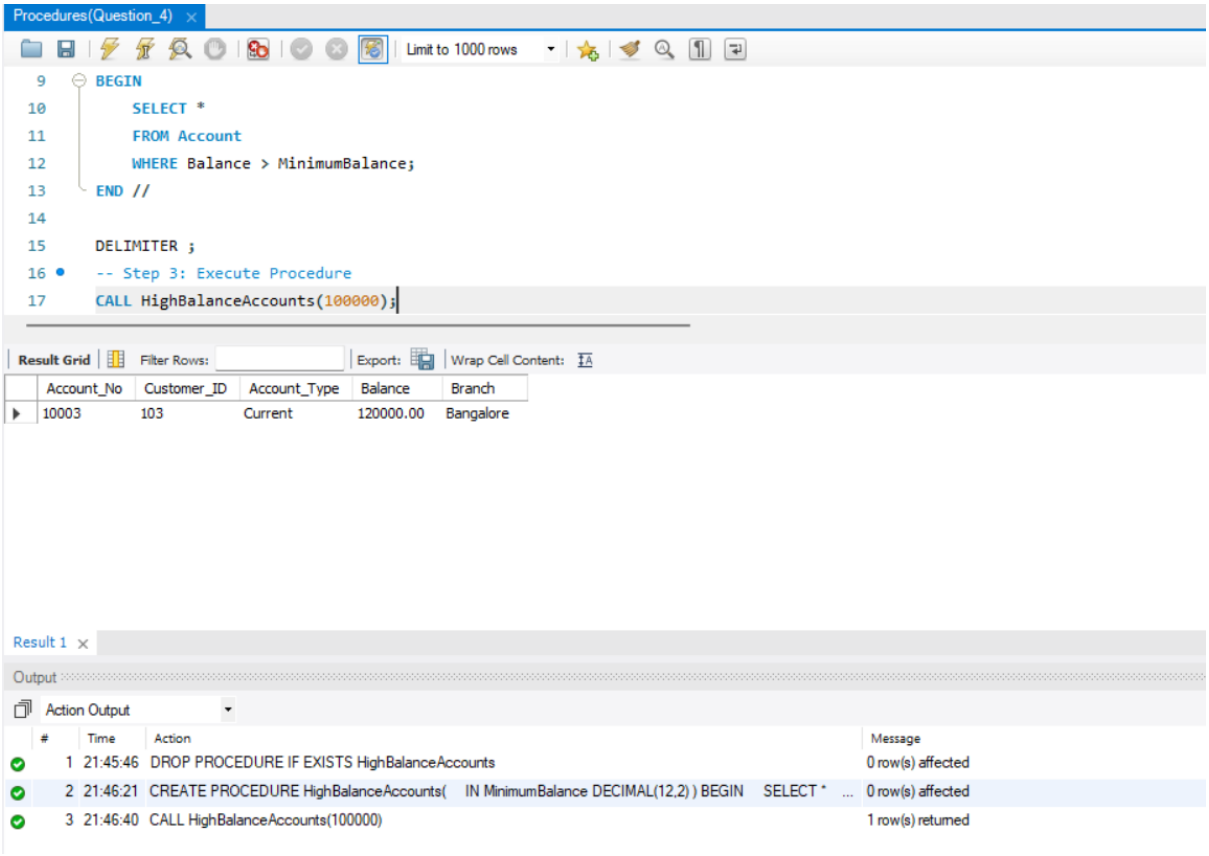
CREATE PROCEDURE HighBalanceAccounts(
    IN MinimumBalance DECIMAL(12,2)
)
BEGIN
    SELECT *
    FROM Account
    WHERE Balance > MinimumBalance;
END //

DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
1	21:45:46	DROP PROCEDURE IF EXISTS HighBalanceAccounts	0 row(s) affected
2	21:46:21	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected



QUESTION 20

Create a procedure to return:

- Account Number
- Customer Name
- Account Type
- Balance
- Branch

for all accounts.

Procedures(Question_20)

```sql
-- Step 1: Remove Existing Procedure
DROP PROCEDURE IF EXISTS GetAllAccountDetails;
-- Step 2: Create Procedure to Display All Account Details
DELIMITER //

CREATE PROCEDURE GetAllAccountDetails()
BEGIN
    SELECT
        A.Account_No,
        C.Customer_Name,
        A.Account_Type,
        A.Balance,
        A.Branch
    FROM Account A
    JOIN Customer C
    ON A.Customer_ID = C.Customer_ID;
END //

DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
1	21:48:49	DROP PROCEDURE IF EXISTS GetAllAccountDetails	0 row(s) affected, 1 warning(s): 1305 PROCEDURE triggers_db.GetAllAccountDetails d
2	21:49:46	CREATE PROCEDURE GetAllAccountDetails() BEGIN SELECT A.Account_No, C.Customer_Name, ...	0 row(s) affected

Procedures(Question_20)

```sql
        A.Branch
    FROM Account A
    JOIN Customer C
    ON A.Customer_ID = C.Customer_ID;
END //

DELIMITER ;
-- Step 3: Execute Procedure
CALL GetAllAccountDetails();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Account_No	Customer_Name	Account_Type	Balance	Branch
10001	Harshika	Savings	47000.00	Hyderabad
10002	Ravinder	Savings	80000.00	Vijayawada
10003	Santhvin	Current	120000.00	Bangalore
10004	Srilatha	Savings	45000.00	Chennai
10005	Varshika	Current	90000.00	Hyderabad

Result 1

Output

Action Output

#	Time	Action	Message
1	21:48:49	DROP PROCEDURE IF EXISTS GetAllAccountDetails	0 row(s) affected, 1 warning(s): 1305 PROCEDURE triggers_db.GetAllAccountDetails does not exist
2	21:49:46	CREATE PROCEDURE GetAllAccountDetails() BEGIN SELECT A.Account_No, C.Customer_Name...	0 row(s) affected
3	21:50:30	CALL GetAllAccountDetails()	5 row(s) returned

CONCEPT : TRIGGERS

QUESTION 5

Create a trigger that prevents an account balance from becoming negative.

Triggers(Question 5)

```
1   -- Step 1: Remove Existing Trigger
2 • DROP TRIGGER IF EXISTS CheckBalance;
```

Output

Action Output

#	Time	Action	Message
1	21:52:25	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected

Triggers(Question 5)

```
1   -- Step 1: Remove Existing Trigger
2 • DROP TRIGGER IF EXISTS CheckBalance;
3   -- Step 2: Create Trigger to Prevent Negative Balance
4   DELIMITER //
5
6 • CREATE TRIGGER CheckBalance
7   BEFORE UPDATE ON Account
8   FOR EACH ROW
9   BEGIN
10      IF NEW.Balance < 0 THEN
11          SIGNAL SQLSTATE '45000'
12          SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
13      END IF;
14  END //
15
16  DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
1	21:52:25	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected
2	21:52:49	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Balanc...	0 row(s) affected

Triggers(Question 5)

```sql
-- Step 1: Remove Existing Trigger
DROP TRIGGER IF EXISTS CheckBalance;
-- Step 2: Create Trigger to Prevent Negative Balance
DELIMITER //

CREATE TRIGGER CheckBalance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
    END IF;
END //

DELIMITER ;
-- Step 3: Test Trigger
UPDATE Account
SET Balance = -1000
WHERE Account_No = 10001;
```

Output

Action Output

#	Time	Action	Message
1	21:52:25	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected
2	21:52:49	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Balanc...	0 row(s) affected
3	21:53:16	UPDATE Account SET Balance = -1000 WHERE Account_No = 10001	Error Code: 1644. Transaction failed: Insufficient balance

Triggers(Question 5)

```sql
DELIMITER ;
-- Step 3: Test Trigger
UPDATE Account
SET Balance = -1000
WHERE Account_No = 10001;
-- Step 4: Verify Account Balance
SELECT *
FROM Account
WHERE Account_No = 10001;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	47000.00	Hyderabad
NULL	NULL	NULL	NULL	NULL

Account 1

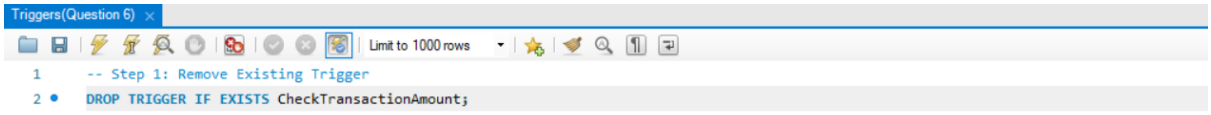
Output

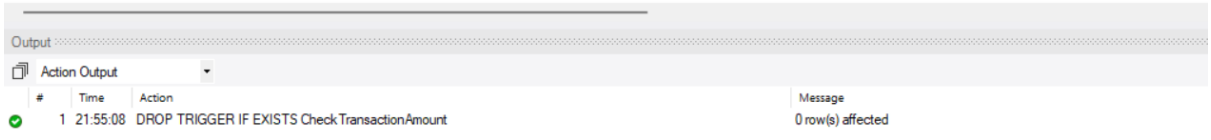
Action Output

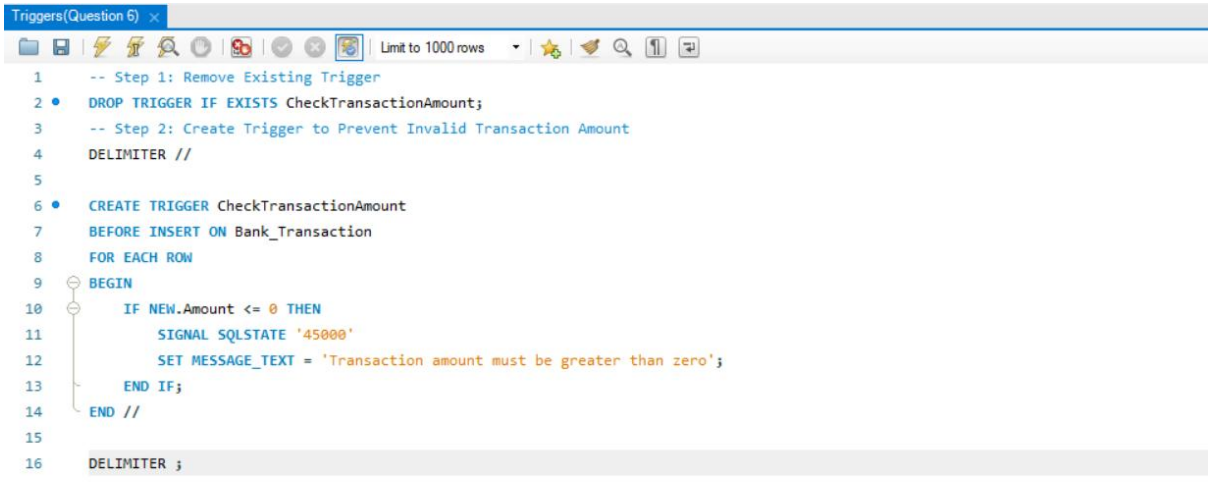
#	Time	Action	Message
1	21:52:25	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected
2	21:52:49	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Balan...	0 row(s) affected
3	21:53:16	UPDATE Account SET Balance = -1000 WHERE Account_No = 10001	Error Code: 1644. Transaction failed: Insufficient balance
4	21:53:36	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000	1 row(s) returned

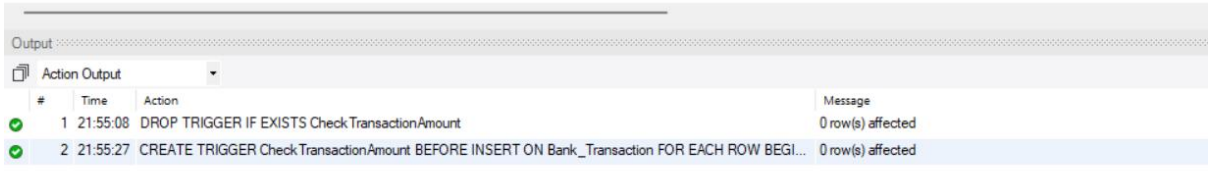
QUESTION 6

Create a trigger that prevents a transaction amount from being zero or negative.









Triggers(Question 6)

```sql
DROP TRIGGER IF EXISTS CheckTransactionAmount;
-- Step 2: Create Trigger to Prevent Invalid Transaction Amount
DELIMITER //

CREATE TRIGGER CheckTransactionAmount
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
    END IF;
END //

DELIMITER ;
-- Step 3: Test Zero Transaction Amount
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 0);
```

Output

Action Output

#	Time	Action	Message
1	21:55:08	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected
2	21:55:27	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BEGI...	0 row(s) affected
3	21:55:48	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 0)	Error Code: 1644. Transaction amount must be greater than zero

Triggers(Question 6)

```sql
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
    END IF;
END //

DELIMITER ;
-- Step 3: Test Zero Transaction Amount
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 0);
-- Step 4: Test Negative Transaction Amount
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', -5000);
```

Output

Action Output

#	Time	Action	Message
1	21:55:08	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected
2	21:55:27	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BEGI...	0 row(s) affected
3	21:55:48	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 0)	Error Code: 1644. Transaction amount must be greater than zero
4	21:56:56	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', -5000)	Error Code: 1644. Transaction amount must be greater than zero